

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · Sistemas Inteligentes

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Vinicius Rodrigues Marquesin

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 03/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	7
4.5	Atividade 5 — Logs e auditoria	9
4.6	Atividade 6 — Upload e perfil de usuário	10
5	Considerações finais	12
6	Declaração de autoria	12

1 Introdução

O presente relatório documenta a evolução técnica e arquitetural do projeto “Cutelaria Marquesin”, desenvolvido ao longo do primeiro bimestre da disciplina. O sistema funciona como um catálogo interativo e seguro onde funcionarios da cutelaria marquesin e clientes da cutelaria podem favoritar, gerenciar e salvar referências visuais de lâminas e facas.

O objetivo do bimestre foi expandir esse escopo simples para simular os desafios de um software de nível de produção. Através das entregas, o sistema deixou de ser um sistema básico para se tornar uma aplicação distribuída, com escalabilidade, segurança e separação de responsabilidades.

Ao longo das semanas, a arquitetura foi transformada com a adoção do padrão de microsserviços. A infraestrutura base foi containerizada utilizando Docker, permitindo a orquestração de múltiplos serviços isolados. O ecossistema atual conta com um frontend reativo em React, um API Gateway (Catálogo API), um microsserviço dedicado de autenticação em Node.js (AuthService), um serviço de mensagens para logs assíncronos via Redis Streams, e um serviço de Object Storage (MinIO) para arquivos binários. Este documento detalha cada etapa dessa construção, comprovando a autoria e a consolidação dos conceitos aplicados.

2 Metodologia

O desenvolvimento do sistema foi pautado em práticas modernas de Engenharia de Software. O ambiente de desenvolvimento local foi utilizando o VS Code e a infraestrutura foi inteiramente orquestrada via docker-compose, garantindo a paridade entre o ambiente de desenvolvimento e produção. O versionamento do código foi realizado através do Git, com repositório remoto hospedado no GitHub. Para a implantação (deploy), utilizou-se o servidor de produção da disciplina (lapps.studio), gerenciado através do Portainer, o que permitiu rotinas de CI/CD (Continuous Integration/Continuous Deployment) baseadas em webhooks e atualizações de imagem (Pull and Redeploy).

A comprovação das atividades descritas neste relatório baseia-se no princípio da rastreabilidade. As datas e horas informadas na seção de entregas (Data Realizada) foram extraídas diretamente dos *timestamps* dos *commits* no histórico do GitHub, obtidos com o comando `git log (git log --date=format:"%d/%m/%Y %H:%M" --format="%h %ad")` e conferidos na página de cada commit no GitHub, garantindo a não alteração dos registros. As evidências visuais (capturas de tela) foram extraídas do ambiente de produção, demonstrando a integração dos microsserviços, a persistência de dados no MariaDB e no MinIO, e o roteamento de rede interna estabelecido no Docker.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	07/08/2026 21:06	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	16/08/2026 19:52	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 20:38	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	03/09/2026 21:02	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	07/09/2026 11:44	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	02/10/2026 23:04	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026 21:06
Situação	entregue
Evidência	Realizada em sala — GitHub, repositório Aula1-Cloud, commit ed9a4dc (07/08/2026 21:06) + print do sistema rodando localmente
Link	https://github.com/VMarquesin/Aula1-Cloud/commit/ed9a4dcb47e5b364fc339f46ef61697e208cfd6f

O que foi feito. Desenvolvimento de uma API fundamental utilizando Python e o microframework Flask. A aplicação consistia em uma agenda telefônica simples para introduzir os conceitos de rotas web, requisições HTTP e manipulação básica de dados em memória ou estruturas simples, servindo como aquecimento para a disciplina.

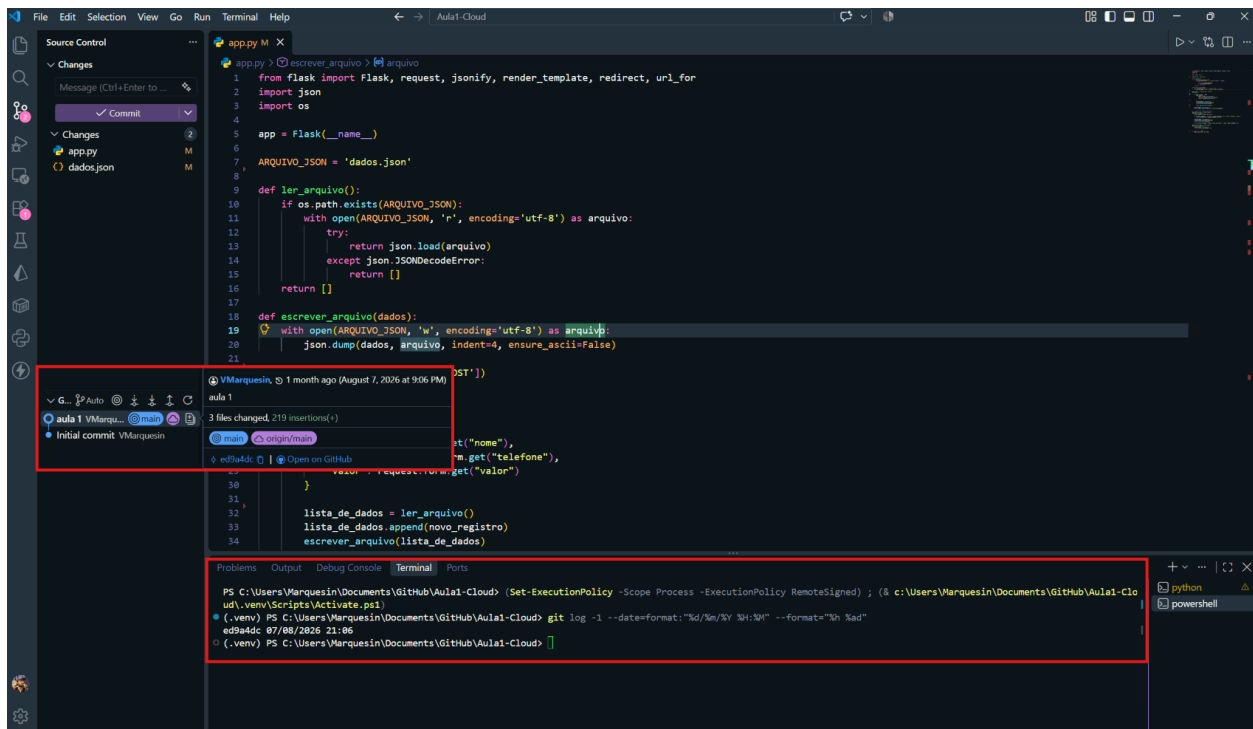


Figura 1: Atividade 1 — Evidência do commit

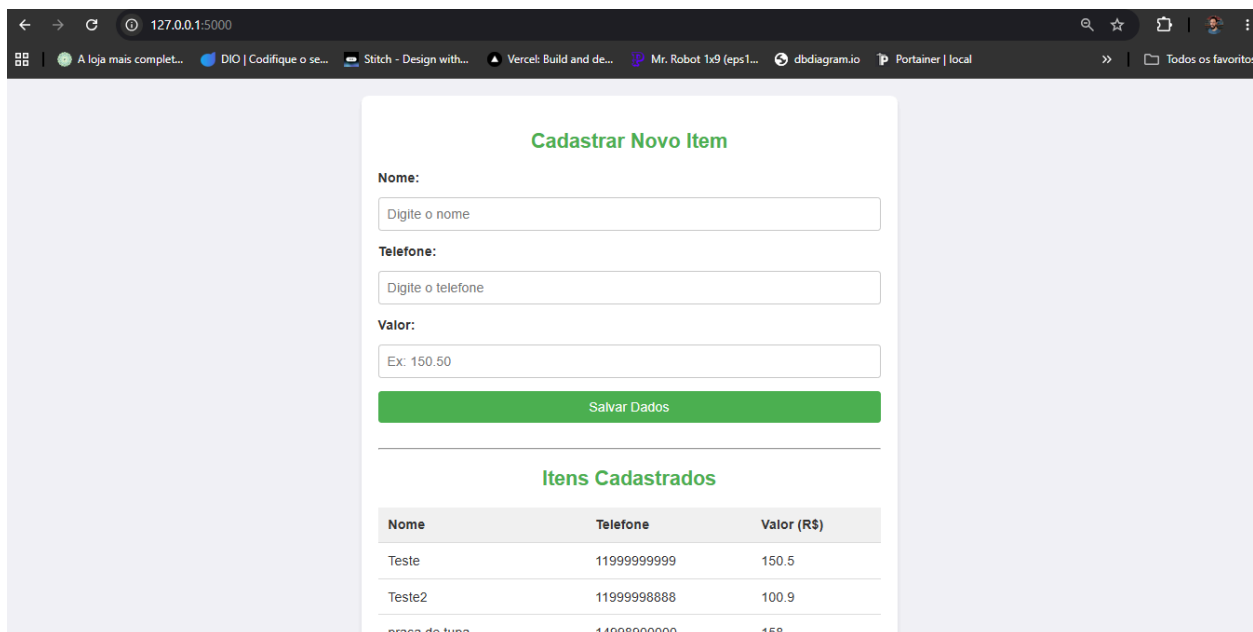


Figura 2: Atividade 1 — Sistema funcionando

Dificuldades e como foram resolvidas. A principal dificuldade inicial foi a configuração do ambiente Python e o entendimento da estrutura de roteamento do Flask. Isso foi superado através da consulta à documentação oficial, uso de IA e do acompanhamento dos exemplos práticos desenvolvidos em sala, melhorando a base para o desenvolvimento de APIs.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026

Data realizada	16/08/2026 19:52
Situação	entregue
Evidência	GitHub — commits e991412 (16/08/2026 13:26) e 9b94195 (16/08/2026 19:52) + README + print do catálogo
Link	https://github.com/VMarquesin/CutelariaMarquesin/commit/e991412f81e88b858d7d8a384700e37869d8ff4f

O que foi feito. O requisito original do catálogo de filmes foi adaptado para o contexto do projeto “Cutelaria Marquesin”. Foi desenvolvida uma aplicação com frontend em React e backend capaz de listar referências de lâminas, conectando-se a APIs externas (como o Unsplash) para buscar imagens, e permitindo ao usuário favoritar e comentar itens.

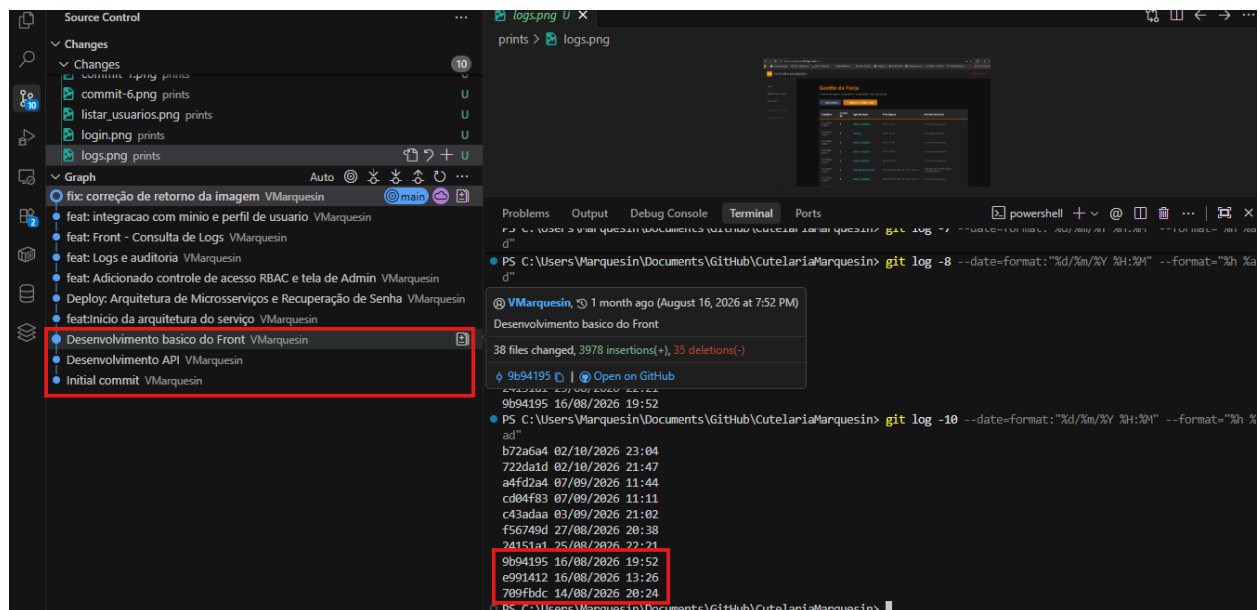


Figura 3: Atividade 2 — Evidência do commit

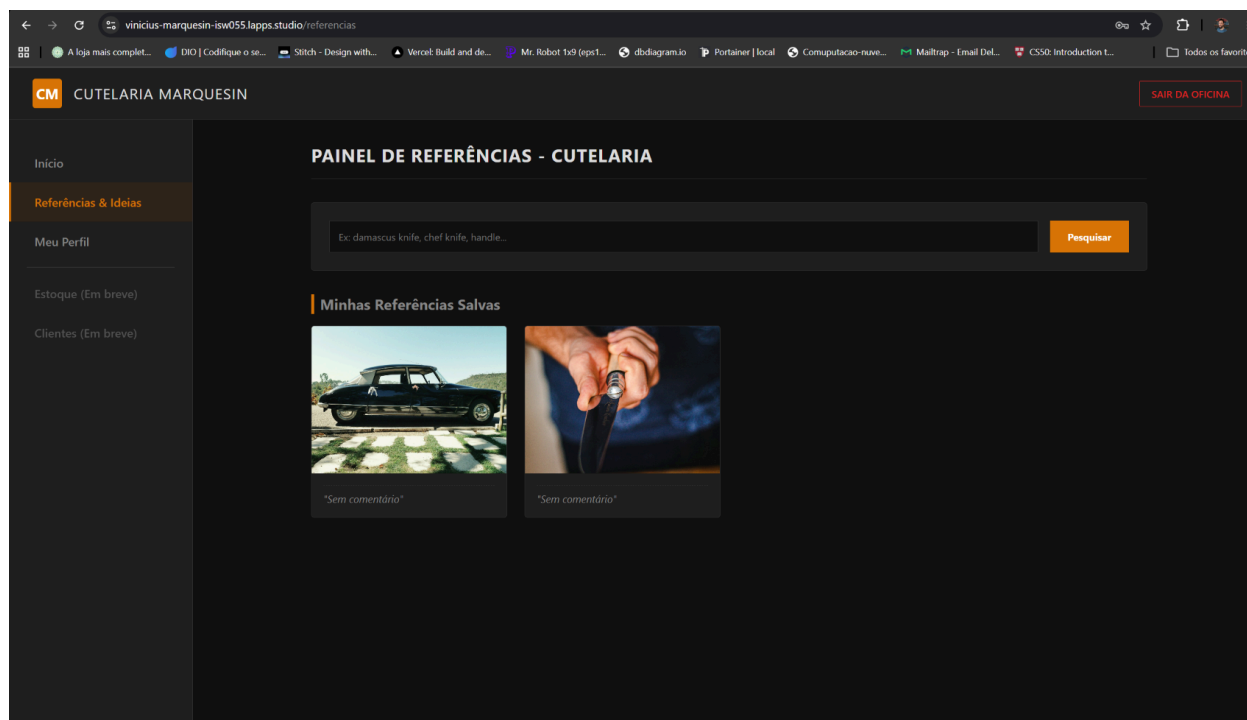


Figura 4: Atividade 2 — Sistema funcionando: painel de referências (catálogo) com itens salvos pelo usuário

Dificuldades e como foram resolvidas. O maior desafio foi estruturar o estado da aplicação no React de forma eficiente e garantir a comunicação correta com a API externa sem expor chaves sensíveis. A solução foi isolar a chamada da API externa no backend (Node.js), servindo os dados tratados para o frontend, garantindo segurança e melhor performance.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 20:38
Situação	entregue
Evidência	GitHub — commits 24151a1 (25/08/2026 22:21) e f56749d (27/08/2026 20:38) + docker-compose.yml + print do login funcionando
Link	https://github.com/VMarquesin/CutelariaMarquesin/commit/24151a15cc0ca5d71d06d03e5d67bc02926c1a25

O que foi feito. Refatoração da arquitetura para a extração do módulo de usuários do monolito original. Foi criado o auth-service, um microsserviço isolado em Node.js responsável exclusivamente por cadastro, login e geração de tokens JWT (JSON Web Tokens). O serviço original do catálogo passou a atuar como um API Gateway, roteando requisições de /auth para o novo serviço.

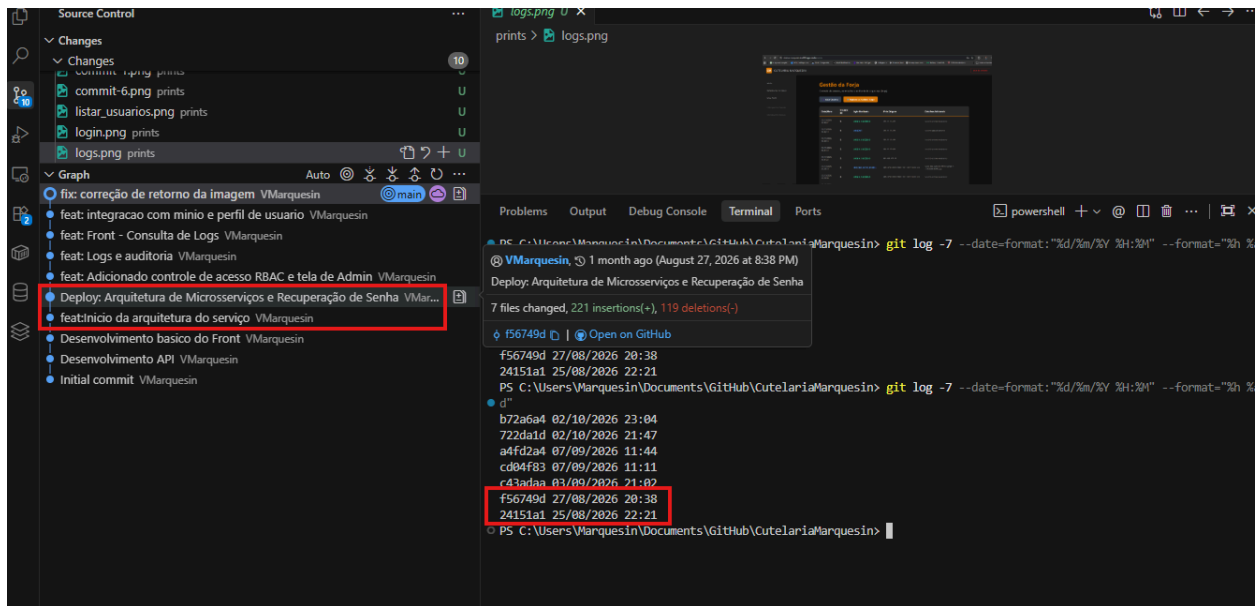


Figura 5: Atividade 3 — Evidência do commit

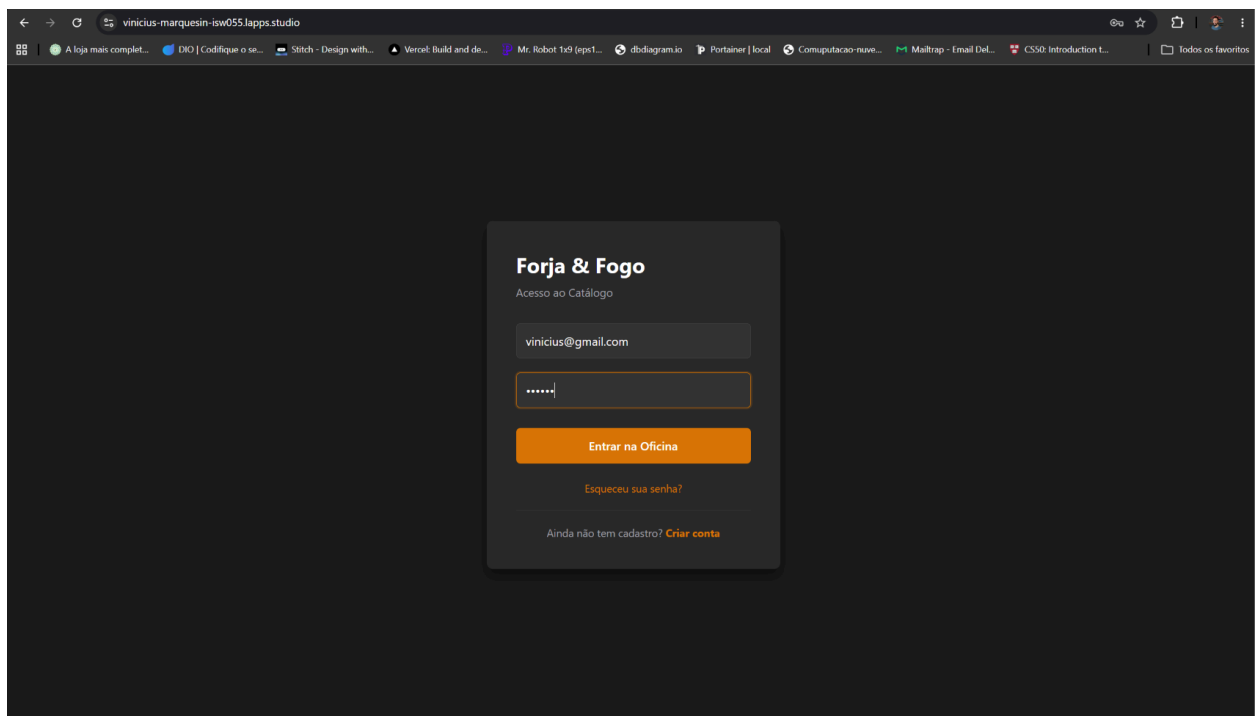


Figura 6: Atividade 3 — Sistema funcionando: tela de login, autenticada pelo auth-service via API Gateway

Dificuldades e como foram resolvidas. A comunicação entre os containers Docker falhava porque o backend tentava acessar localhost em vez da rede interna. O problema foi resolvido configurando a rede bridge no docker-compose.yml e utilizando o nome do serviço (auth-service:3001) como resolução de DNS interno para as requisições do proxy.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026

Data realizada	03/09/2026 21:02
Situação	entregue
Evidência	GitHub — commit c43adaa (03/09/2026 21:02) + print do 403 e da ação de admin
Link	https://github.com/VMarquesin/CutelariaMarquesin/commit/f56749d53bc73627e0c46db2473e904ca40cb5c9

O que foi feito. Implementação do modelo *Role-Based Access Control* (Padrão B — Claims no JWT). A tabela de usuários foi atualizada para suportar perfis (roles) como ‘admin’. O JWT gerado pelo auth-service passou a embutir essa *claim*. Foram criados middlewares no backend para interceptar requisições críticas, garantindo que apenas administradores pudessem acessar rotas sensíveis (gerando erro 403 Forbidden para acessos indevidos).

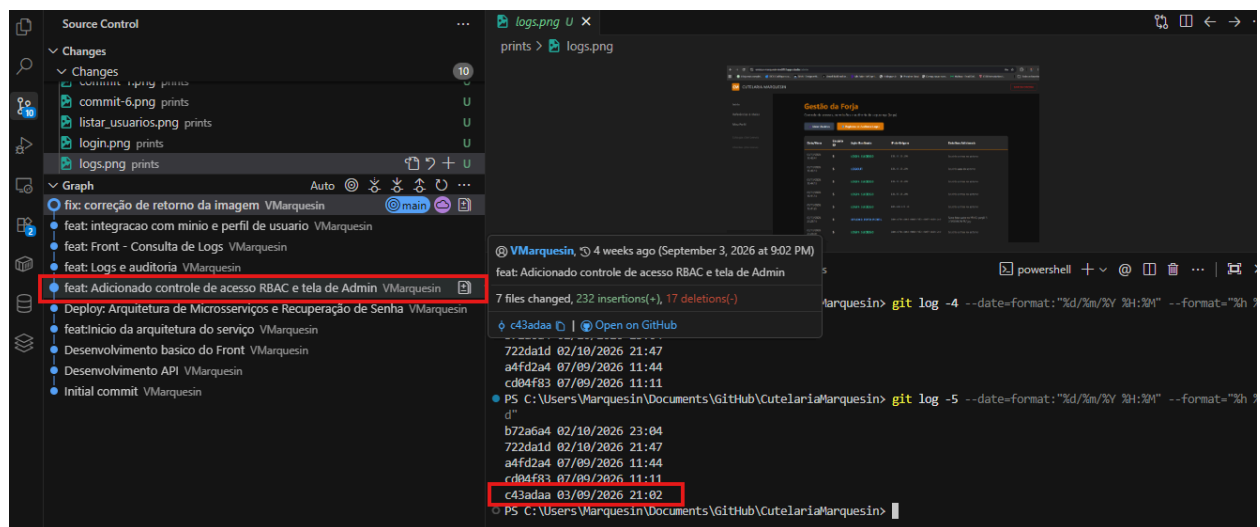


Figura 7: Atividade 4 — Evidência do commit

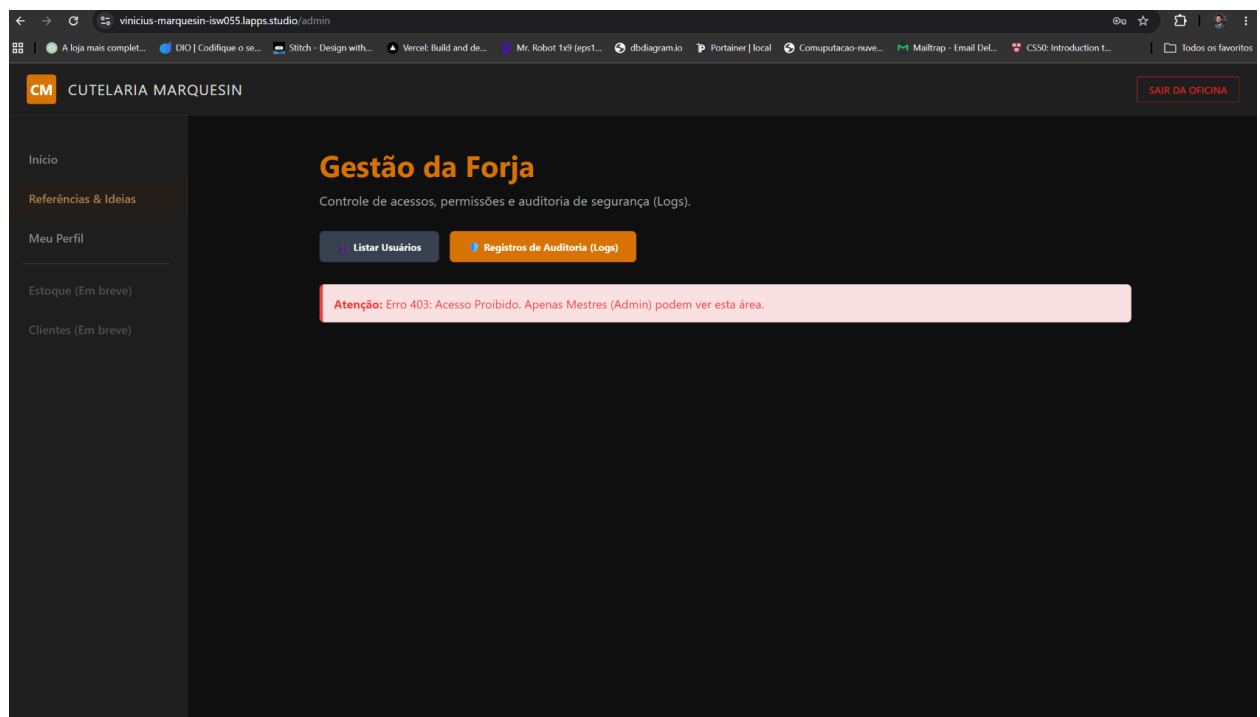


Figura 8: Atividade 4 — Sistema funcionando: usuário comum recebe 403 ao acessar área restrita

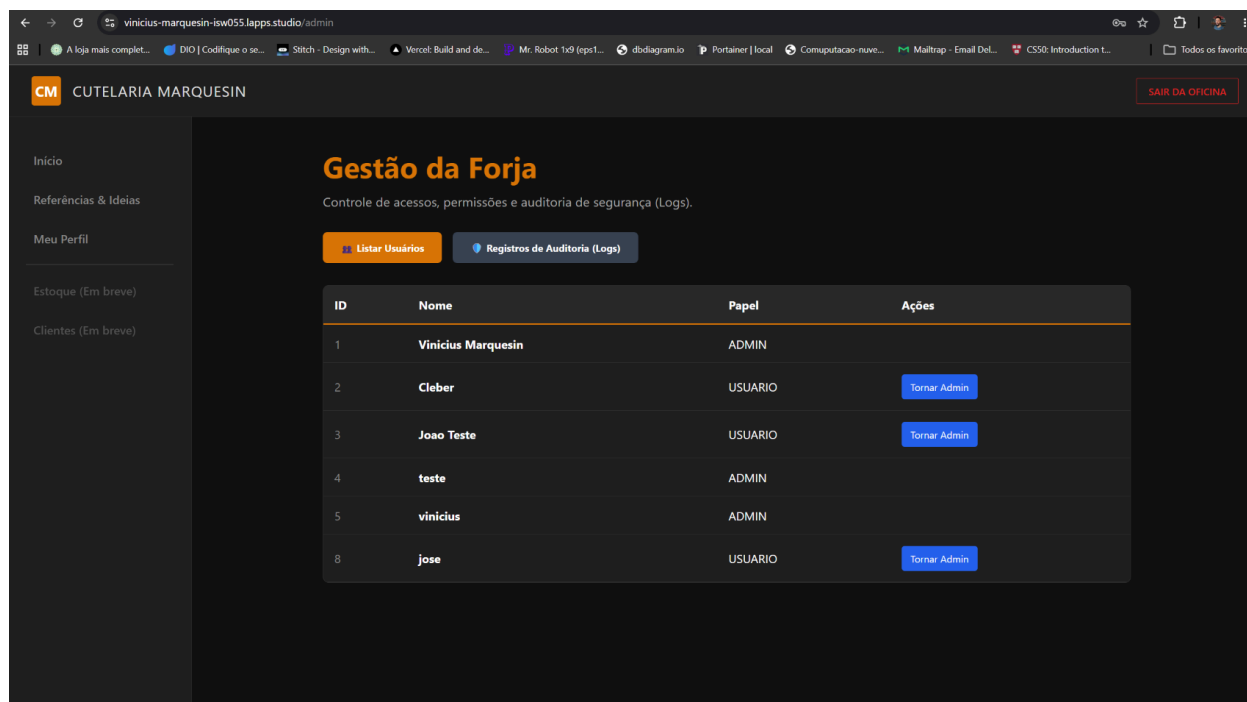


Figura 9: Atividade 4 — Sistema funcionando: ação de admin (listagem de usuários e promoção de papel)

Dificuldades e como foram resolvidas. Garantir que o cabeçalho de Authorization (Bearer Token) fosse repassado corretamente pelo API Gateway sem ser descartado. Foi necessário ajustar a configuração do proxy reverso no Catálogo para preservar os *headers* originais da requisição do cliente até o serviço de autenticação.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	07/09/2026 11:44
Situação	entregue
Evidência	GitHub — commits cd04f83 (07/09/2026 11:11) e a4fd2a4 (07/09/2026 11:44) + print da consulta de logs pelo admin
Link	https://github.com/VMarquesin/CutelariaMarquesin/commit/cd04f83ebb690d07b16616037c50ead9aa5f917e

O que foi feito. Implementação de uma arquitetura orientada a eventos (Producer/Consumer) para observabilidade. Foi inserido um container do Redis atuando como fila de mensagens (Redis Streams). O auth-service e o catalogo-api foram configurados como produtores, disparando logs de ações (logins, erros, favoritos). Um novo microserviço (log-service) foi criado como consumidor assíncrono para ler a fila e persistir a auditoria para visualização em um painel administrativo no frontend.

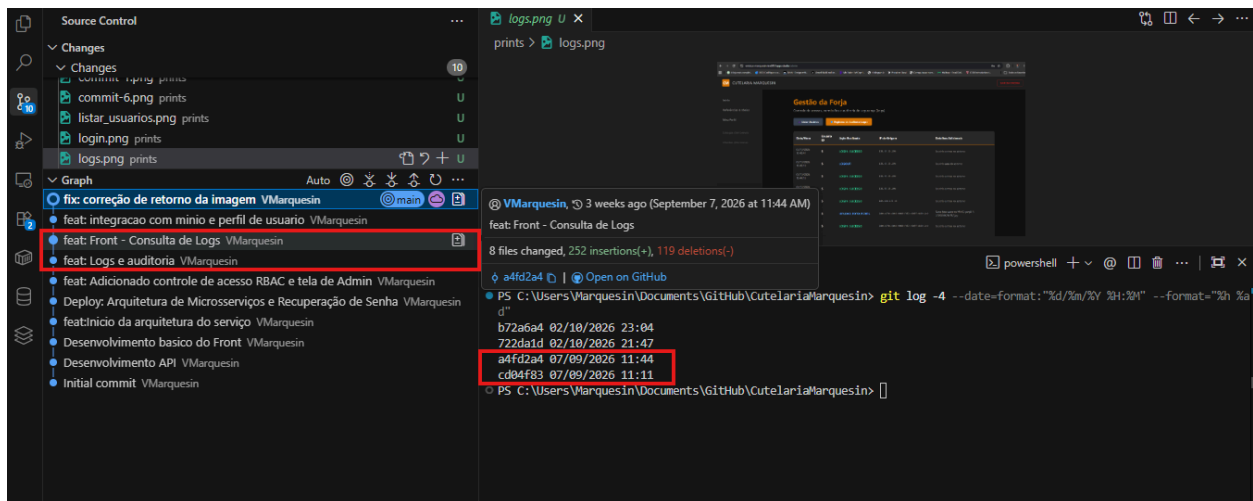


Figura 10: Atividade 5 — Evidência do commit

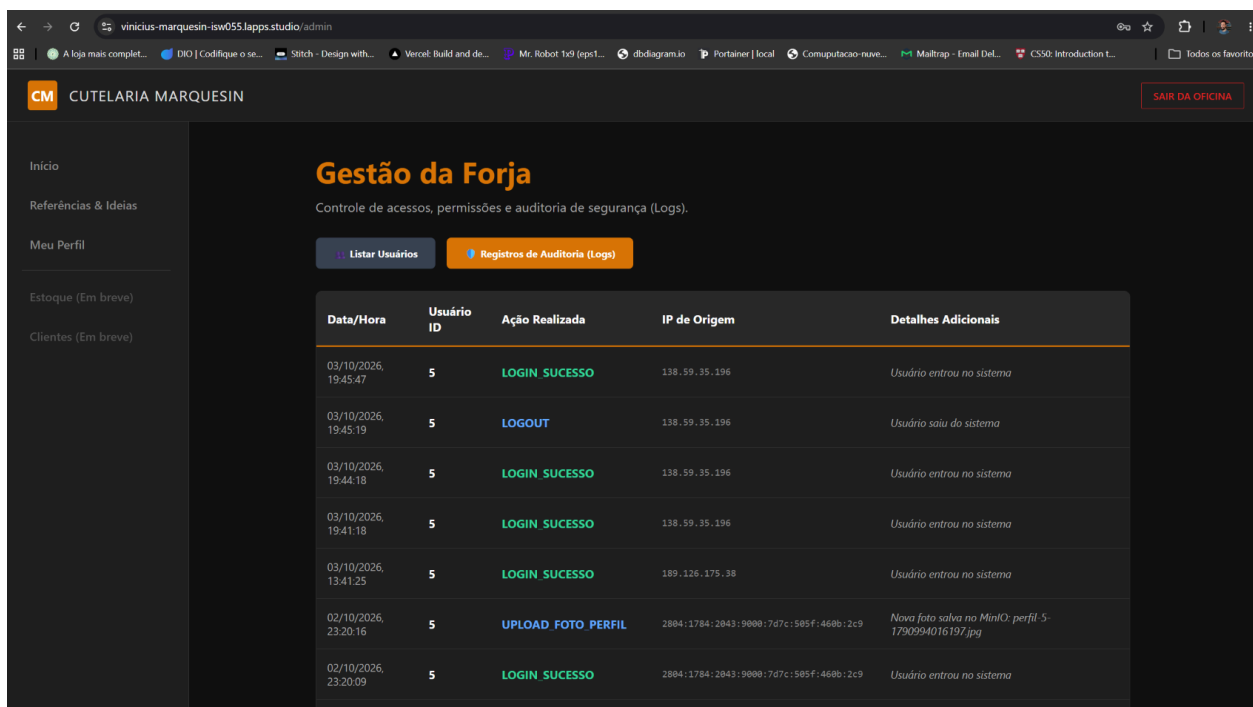


Figura 11: Atividade 5 — Sistema funcionando: consulta dos registros de auditoria pelo admin

Dificuldades e como foram resolvidas. Assegurar que a geração de logs não causasse lentidão ou gargalo (Time-out) na resposta principal ao usuário. A solução arquitetural foi a adoção do Redis Streams, permitindo operações de escrita (XADD).

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	02/10/2026 23:04
Situação	entregue
Evidência	GitHub — commits 722da1d (02/10/2026 21:47) e b72a6a4 (02/10/2026 23:04) + print do perfil com foto

Link	https://github.com/VMarquesin/CutelariaMarquesin/commit/722da1d26aee656ddd3f8ccbfcddec
-------------	---

O que foi feito. Transição do armazenamento de arquivos de uma abordagem relacional para Object Storage S3-Compatible utilizando o MinIO (pgsty/minio). O container foi configurado na rede privada (sem portas expostas externamente). O upload processa imagens em multipart/form-data via memória (multer) e envia os binários ao MinIO, gravando apenas a URL de referência no MariaDB. Foi implementado o bloqueio anti-IDOR, garantindo que a edição do perfil dependa estritamente do ID do JWT, e não do corpo da requisição.

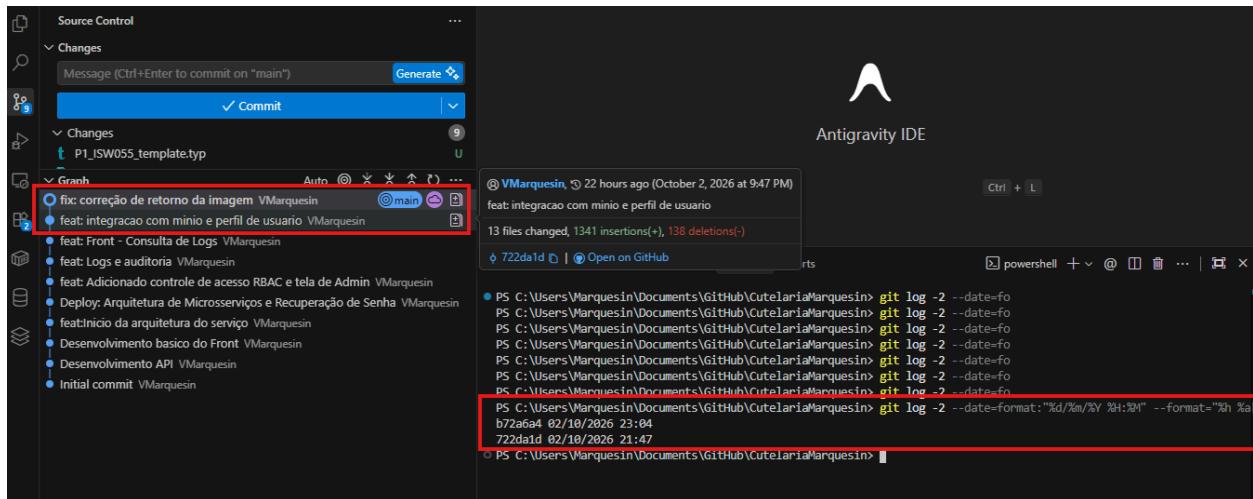


Figura 12: Atividade 6 — Evidência do commit

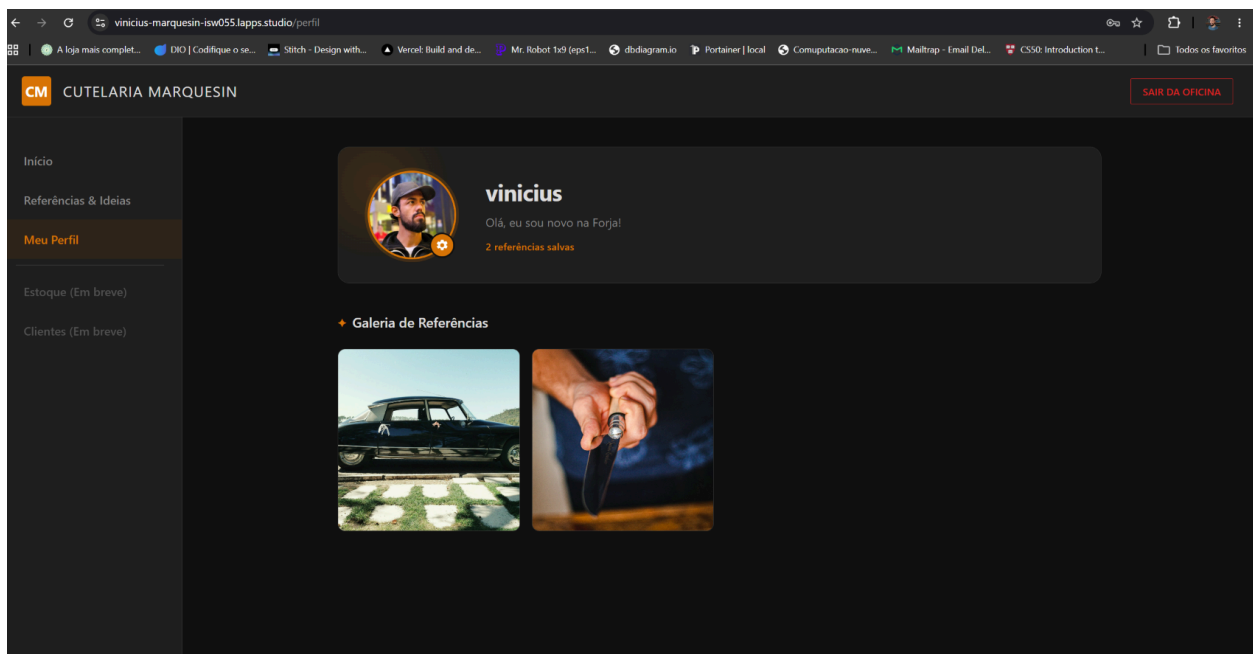


Figura 13: Atividade 6 — Sistema funcionando: perfil com foto armazenada no MinIO

Dificuldades e como foram resolvidas. O principal desafio técnico foi o corrompimento do binário da imagem pelo API Gateway (Catálogo), que tentava desserializar a requisição como JSON, além do bloqueio de portas públicas. A solução exigiu que o MinIO fosse isolado internamente, e as rotas de proxy de ida e volta foram reescritas para utilizar fluxos de rede nativos (`req.pipe(proxyReq)` e `responseType: 'stream'`), garantindo que o *buffer* binário viajasse intacto entre o navegador, o proxy e o AuthService.

5 Considerações finais

O desenvolvimento deste projeto consolidou uma transição fundamental: a passagem da escrita de código básico para o desenho de arquiteturas de sistemas distribuídos. Mais do que aprender a sintaxe de uma linguagem, o bimestre exigiu a compreensão de redes Docker, comunicação inter-processos, gerenciamento de estado entre microsserviços e segurança em trânsito (JWT, RBAC). Lidar com a infraestrutura diretamente no Portainer trouxe uma visão dos desafios de DevOps.

A maior lição técnica do período foi sobre arquitetura e a natureza dos protocolos de rede. Problemas como conflitos de portas em servidores compartilhados e a corrupção de *streams* binários por proxies mal configurados (durante a integração com o MinIO) provaram que um sistema escalável exige decisões de design (como o isolamento do Object Storage sem exposição pública). Hoje, a Cutelaria Marquesin possui uma fundação mais sólida e estruturada para crescer de maneira segura e independente.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Vinicius Rodrigues Marquesin

Pompeia, 03/10/2026